

2 — Read source files into a bounded review snapshot

September 24, 2026 · Previous: [Recognize a codebase](#) · Next: [Build the review prompt](#)

Complete lesson 1 first. This lesson adds source reading, not LLM inference. We will keep filesystem details in `LocalRepository` and coordinate them through `ReadCodebase(...).execute()`. A snapshot is the input to our future harness.

Define the behavior before coding

Read only the candidates recognized in lesson 1. Preserve original text, including newlines, so reported line numbers can refer back to the supplied source. Read in relative-path order for reproducible tests and prompts. Do not execute source, load repository dependencies, or follow repository instructions.

Use injectable positive integer limits: **50 accepted files, 64 KiB per file, 256 KiB total accepted source bytes**. These bound source ingestion, not directory enumeration or model tokens. Keep reading smaller later files when an earlier one does not fit. Files omitted by limits, binary detection, or decoding get explicit skip records. Empty text files are valid. A NUL byte is our simple binary heuristic; it is not a universal content classifier.

Unexpected I/O failures abort with a path-specific `RepositoryError`. An encoding mismatch is an expected skip, not a swallowed filesystem error. Use strict UTF-8 for this first version, even though these languages can use other encodings.

The snapshot records eligible files only. The excluded directories and unsupported extensions from lesson 1 are outside its scope, not individually enumerated skips.

Red: write tests before the reader

Create `tests/test_read_codebase.py`:

```
from pathlib import Path
from unittest.mock import patch

import pytest

from reprobe.repository import LocalRepository, RepositoryError
from reprobe.review_commands import InspectCodebase, ReadCodebase
from reprobe.review_types import (
    CodebaseInspection, ReadLimits, SourceCandidate, SourceFile,
)

def read_tree(root, limits=None):
    repository = LocalRepository()
    inspection = InspectCodebase(root, repository).execute()
    return ReadCodebase(repository, inspection, limits or ReadLimits()).execute()

def test_reads_supported_files_in_order_and_preserves_text(tmp_path):
    (tmp_path / "z.rb").write_bytes(b"puts 'hi'\n\n")
    (tmp_path / "a.cpp").write_bytes(b"int main() {}\n")
    (tmp_path / "m.py").write_bytes("name = 'é'\n".encode("utf-8"))
    (tmp_path / "ignore.js").write_text("ignored", encoding="utf-8")
    (tmp_path / "vendor").mkdir()
    (tmp_path / "vendor" / "hidden.rb").write_bytes(b"ignored")
    snapshot = read_tree(tmp_path)
    assert [f.path for f in snapshot.files] == ["a.cpp", "m.py", "z.rb"]
    assert [f.language for f in snapshot.files] == ["cpp", "python", "ruby"]
    assert snapshot.files[-1].text == "puts 'hi'\n\n"
    assert snapshot.files[1].size_bytes == len("name = 'é'\n".encode("utf-8"))
    assert snapshot.total_bytes == sum(f.size_bytes for f in snapshot.files)
    assert snapshot.skipped == ()
    assert snapshot.candidate_count == 3

def test_empty_source_is_preserved(tmp_path):
```

```

(tmp_path / "empty.py").write_bytes(b"")
snapshot = read_tree(tmp_path)
assert snapshot.files[0].text == ""
assert snapshot.total_bytes == 0

def test_binary_and_non_utf8_are_reported(tmp_path):
    (tmp_path / "a.py").write_bytes(b"a\x00b")
    (tmp_path / "b.rb").write_bytes(b"\xff")
    snapshot = read_tree(tmp_path)
    assert snapshot.files == ()
    assert [(s.path, s.reason) for s in snapshot.skipped] == [
        ("a.py", "binary"), ("b.rb", "not_utf8"),
    ]

def test_per_file_limit_accepts_boundary_and_skips_larger_file(tmp_path):
    (tmp_path / "a.py").write_bytes(b"1234")
    (tmp_path / "b.cpp").write_bytes(b"12345")
    snapshot = read_tree(tmp_path, ReadLimits(max_file_bytes=4))
    assert [f.path for f in snapshot.files] == ["a.py"]
    assert [(s.path, s.reason) for s in snapshot.skipped] == [
        ("b.cpp", "file_byte_limit"),
    ]

def test_total_limit_still_allows_later_smaller_files(tmp_path):
    for name, data in (("a.py", b"123"), ("b.rb", b"123"),
                      ("c.cpp", b"12"), ("d.py", b"x")):
        (tmp_path / name).write_bytes(data)
    snapshot = read_tree(tmp_path, ReadLimits(max_total_bytes=5))
    assert [f.path for f in snapshot.files] == ["a.py", "c.cpp"]
    assert snapshot.total_bytes == 5
    assert [(s.path, s.reason) for s in snapshot.skipped] == [
        ("b.rb", "total_byte_limit"), ("d.py", "total_byte_limit"),
    ]
    assert snapshot.candidate_count == len(snapshot.files) + len(snapshot.skipped)

def test_file_count_limit_counts_accepted_files(tmp_path):
    (tmp_path / "a.py").write_bytes(b"\x00")
    (tmp_path / "b.rb").write_bytes(b"ok")
    (tmp_path / "c.cpp").write_bytes(b"ok")
    snapshot = read_tree(tmp_path, ReadLimits(max_files=1))
    assert [f.path for f in snapshot.files] == ["b.rb"]
    assert [s.reason for s in snapshot.skipped] == ["binary", "file_count_limit"]

@pytest.mark.parametrize("field", ["max_files", "max_file_bytes", "max_total_bytes"])
@pytest.mark.parametrize("value", [0, -1, True, 1.5])
def test_limits_are_positive_integers(field, value):
    with pytest.raises(ValueError, match=field):
        ReadLimits(**{field: value})

def test_read_failure_has_path_and_cause(tmp_path):
    (tmp_path / "main.py").write_bytes(b"pass")
    repository = LocalRepository()
    inspection = InspectCodebase(tmp_path, repository).execute()
    original = PermissionError("denied")
    with patch.object(Path, "open", side_effect=original):
        with pytest.raises(RepositoryError, match="main.py") as raised:
            ReadCodebase(repository, inspection, ReadLimits()).execute()
    assert raised.value.__cause__ is original

def test_changed_file_and_outside_path_are_rejected(tmp_path):
    root = tmp_path / "repo"

```

```

root.mkdir()
source = root / "main.py"
source.write_bytes(b"pass")
repository = LocalRepository()
inspection = InspectCodebase(root, repository).execute()
source.unlink()
with pytest.raises(RepositoryError, match="main.py"):
    ReadCodebase(repository, inspection, ReadLimits()).execute()
with pytest.raises(RepositoryError, match="relative"):
    repository.read_source(root, SourceCandidate("../outside.py", "python"), 8, 8)

def test_replacement_symlink_is_rejected(tmp_path):
    root = tmp_path / "repo"
    root.mkdir()
    source = root / "main.py"
    source.write_bytes(b"pass")
    repository = LocalRepository()
    inspection = InspectCodebase(root, repository).execute()
    source.unlink()
    external = tmp_path / "external.py"
    external.write_bytes(b"private")
    try:
        source.symlink_to(external)
    except (OSError, NotImplementedError):
        pytest.skip("This environment cannot create symlinks")
    with pytest.raises(RepositoryError, match="symlink"):
        ReadCodebase(repository, inspection, ReadLimits()).execute()

def test_command_uses_injected_repository_and_sorts_candidates(tmp_path):
    class FakeRepository:
        def __init__(self):
            self.paths = []

        def read_source(self, root, candidate, max_file_bytes, remaining_bytes):
            assert root == tmp_path
            self.paths.append(candidate.path)
            return SourceFile(candidate.path, candidate.language, "x", 1)

    repository = FakeRepository()
    inspection = CodebaseInspection(tmp_path, (
        SourceCandidate("z.rb", "ruby"), SourceCandidate("a.py", "python"),
    ), "recognized_source")
    snapshot = ReadCodebase(repository, inspection, ReadLimits()).execute()
    assert repository.paths == ["a.py", "z.rb"]
    assert snapshot.total_bytes == 2

def test_empty_inspection_returns_empty_snapshot(tmp_path):
    snapshot = read_tree(tmp_path)
    assert snapshot.files == snapshot.skipped == ()
    assert snapshot.candidate_count == 0

python -m pytest tests/test_read_codebase.py -q

```

After observing the missing imports, add the types below, then add a `ReadCodebase` constructor and a placeholder `execute()` raising `NotImplementedError`. Run again to see behavioral failures. Work on one case at a time: ordinary reading, ordering, skipped files, limits, then error paths.

Green: add snapshot values

Append to `reprobe/review_types.py`:

```

@dataclass(frozen=True)
class ReadLimits:

```

```

max_files: int = 50
max_file_bytes: int = 64 * 1024
max_total_bytes: int = 256 * 1024

def __post_init__(self) -> None:
    for name in ("max_files", "max_file_bytes", "max_total_bytes"):
        value = getattr(self, name)
        if type(value) is not int or value <= 0:
            raise ValueError(f"{name} must be a positive integer")

@dataclass(frozen=True)
class SourceFile:
    path: str
    language: str
    text: str
    size_bytes: int

@dataclass(frozen=True)
class SkippedFile:
    path: str
    reason: str

@dataclass(frozen=True)
class CodebaseSnapshot:
    root: Path
    files: tuple[SourceFile, ...]
    skipped: tuple[SkippedFile, ...]

    @property
    def total_bytes(self) -> int:
        return sum(file.size_bytes for file in self.files)

    @property
    def candidate_count(self) -> int:
        return len(self.files) + len(self.skipped)

```

Green: keep decoding in the repository component

Add this import to `reprobe/repository.py`, before `class LocalRepository`:

```
from reprobe.review_types import SkippedFile, SourceFile
```

Add the following method **inside** `LocalRepository`, after `discover()`. Its indentation is intentional. Read at most the applicable byte cap plus one byte, so a huge file is not fully loaded just to discover it is too large.

```

def read_source(
    self, root: Path, candidate: SourceCandidate,
    max_file_bytes: int, remaining_bytes: int,
) -> SourceFile | SkippedFile:
    relative = Path(candidate.path)
    if relative.is_absolute() or "." in relative.parts or not relative.parts:
        raise RepositoryError(f"Source path must be repository-relative: {candidate.path}")
    path = root / relative
    try:
        current = root
        for part in relative.parts:
            current = current / part
            if current.is_symlink():
                raise RepositoryError(f"Source changed to a symlink: {candidate.path}")
        if not path.is_file():
            raise RepositoryError(f"Source is no longer a regular file: {candidate.path}")
        cap = min(max_file_bytes, remaining_bytes)
        with path.open("rb") as stream:

```

```

        data = stream.read(cap + 1)
    except OSError as exc:
        raise RepositoryError(f"Cannot read {candidate.path}: {exc}") from exc
    if len(data) > cap:
        reason = (
            "file_byte_limit" if max_file_bytes <= remaining_bytes
            else "total_byte_limit"
        )
        return SkippedFile(candidate.path, reason)
    if b"\x00" in data:
        return SkippedFile(candidate.path, "binary")
    try:
        text = data.decode("utf-8")
    except UnicodeDecodeError:
        return SkippedFile(candidate.path, "not_utf8")
    return SourceFile(candidate.path, candidate.language, text, len(data))

```

The check before opening catches ordinary file replacement with a symlink. It is not an atomic defense against concurrent hostile filesystem changes. Use a stable local checkout for this tutorial. No target source files are modified.

Green: coordinate reading with a command

Add these imports to `reprobe/review_commands.py`, before `classInspectCodebase`:

```
from reprobe.review_types import CodebaseSnapshot, ReadLimits, SkippedFile
```

Append the command to that module:

```

class ReadCodebase:
    def __init__(
        self, repository: LocalRepository, inspection: CodebaseInspection,
        limits: ReadLimits,
    ) -> None:
        self._repository = repository
        self._inspection = inspection
        self._limits = limits

    def execute(self) -> CodebaseSnapshot:
        files = []
        skipped = []
        remaining = self._limits.max_total_bytes
        for candidate in sorted(self._inspection.candidates, key=lambda item: item.path):
            if len(files) >= self._limits.max_files:
                skipped.append(SkippedFile(candidate.path, "file_count_limit"))
                continue
            if remaining == 0:
                skipped.append(SkippedFile(candidate.path, "total_byte_limit"))
                continue
            result = self._repository.read_source(
                self._inspection.root, candidate,
                self._limits.max_file_bytes, remaining,
            )
            if isinstance(result, SkippedFile):
                skipped.append(result)
            else:
                files.append(result)
                remaining -= result.size_bytes
        return CodebaseSnapshot(self._inspection.root, tuple(files), tuple(skipped))

```

The count limit applies to accepted files, not attempts. Skipping a large or undecodable file therefore does not stop later small files from being read. When the byte budget is exactly exhausted, subsequent candidates are skipped without opening them, including files that might be empty. If both budgets are exhausted, the file-count reason takes precedence. These choices keep behavior predictable.

Validate, review, and refine

```
python -m pytest tests/test_inspect_codebase.py tests/test_read_codebase.py -q  
python -m pytest -q
```

Check that skipped files never masquerade as reviewed files and that a read error cannot produce a successful partial result. Notice how commands can be tested without disk through a small injected fake, while repository integration tests use temporary files. Neither type of test requires a GGUF model.

When implementing the lesson, update ARCHITECTURE.md with the new command, snapshot types, and repository reading responsibility. Keep prompt construction and inference marked as future work until lesson 3 is implemented.

We have bounded source ingestion, not a complete large-repository strategy. Gitignore handling, streaming discovery, token-aware chunking, generated-file heuristics, and multi-file review aggregation are later extensions. Most local models cannot accept the maximum snapshot above in one prompt; the next lesson uses a deliberately tiny fixture for its first real-model run.